



GABRYELLA SANTOS PINHO
LOUIE NERY SILVA
MARIA EDUARDA RITA MARQUES NOLETO
NATHANAEL VICTOR PAIVA MAGNO
RAMON MIGUEL ROSA PEREIRA ATAIDES

APÊNDICE

Artigo apresentado ao curso de graduação em Análise e Desenvolvimento de Sistemas da Universidade Católica de Brasília, como requisito parcial para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Orientador: William Roberto Malvezzi

Brasília
2026

Apêndice A – Dicionário de Atributos (Spotify Audio Features)

Este dicionário descreve as variáveis técnicas extraídas da API do Spotify, que serviram como preditores para a classificação dos géneros musicais.

Atributo	Descrição Técnica	Intervalo
Acousticness	Medida de confiança se a faixa é acústica.	0.0 a 1.0
Danceability	Adequação da faixa para dançar (ritmo, estabilidade, batida).	0.0 a 1.0
Energy	Medida de intensidade e atividade (rápida, alta e ruidosa).	0.0 a 1.0
Instrumentalness	Previsão de que a faixa não contém vocais.	0.0 a 1.0
Liveness	Detecta a presença de uma audiência na gravação (efeito "ao vivo").	0.0 a 1.0
Loudness	O volume geral da faixa em decibéis (dB).	-60 a 0 dB
Speechiness	Presença de palavras faladas na faixa.	0.0 a 1.0
Valence	Positividade musical (alegria, euforia ou tristeza, raiva).	0.0 a 1.0
Tempo	A velocidade estimada em batidas por minuto (BPM).	0 a 200+
Popularity	Algoritmo do Spotify baseado no número de reproduções recentes.	0 a 100

Apêndice B – Protocolo Experimental e Reprodutibilidade

Para garantir que a comparação entre as linguagens R e Python fosse estatisticamente válida, foi estabelecido o seguinte protocolo:

- **Semente Aleatória (Seed):** Fixada em **42** em ambos os ambientes (`random_state=42` no Python e `set.seed(42)` no R).
- **Divisão de Dados:** 80% para treinamento e 20% para teste, utilizando amostragem estratificada para manter a proporção de gêneros.
- **Tratamento de Dados:**
 - Remoção de duplicatas baseada no `track_id`.
 - Tratamento de valores ausentes via *imputation* (mediana para numéricos, moda para categóricos).
 - Criação de variáveis derivadas: `duration_minutes` (de milissegundos) e `release_year`.
- **Escalação:** Aplicação de normalização (StandardScaler) para algoritmos sensíveis à escala (especialmente no pipeline Python).

Apêndice C – Estrutura Lógica de Implementação

Abaixo, descreve-se a arquitetura das ferramentas utilizadas:

- **Ambiente Python:** Utilização da biblioteca `scikit-learn` com o recurso de `Pipeline`, garantindo que o pré-processamento (OneHotEncoder para subgêneros e SimpleImputer) fosse aplicado de forma isolada no treino e replicado no teste sem vazamento de dados (*data leakage*).
- **Ambiente R:** Utilização do ecossistema `tidyverse` para manipulação e das bibliotecas `caret`, `rpart`, `ubridate`, `dplyr`, `jsonlite`, `naivebayes` e `randomForest` para modelagem estatística.

Apêndice D – Tabela Comparativa Final de Resultados

A tabela abaixo consolida o desempenho dos seis modelos treinados. As métricas referem-se aos testes realizados no conjunto de validação (20% da base).

Tabela: Comparação Global de Performance (R vs. Python)

Linguagem	Algoritmo	Acurácia	Precisão (Média)	Recall (Média)	F1-Score
Python	Naive Bayes	80%	87%	80%	81%
Python	Árvore de Decisão	25%	54%	25%	26%
Python	Random Forest	76%	81%	76%	77%
R	Naive Bayes	72%	65%	66%	66%
R	Árvore de Decisão	31%	33%	13%	38%
R	Random Forest	81%	82%	60%	70%

Nota de Análise: Conforme discutido na metodologia, o modelo **Random Forest** apresentou a maior robustez em ambas as linguagens devido à sua capacidade de lidar com a não-linearidade dos atributos sonoros e a complexidade dos subgêneros musicais. O Naive Bayes, embora mais rápido, foi penalizado pela premissa de independência entre atributos que, na música (como *Energy* e *Loudness*), são altamente correlacionados.

Apêndice E – Requisitos de Software

Para execução dos códigos anexos, são necessários:

- **Python 3.10+:** pandas, numpy, scikit-learn, joblib.
- **R 4.2+:** tidyverse, caret, randomForest, naivebayes, rpart, lubridate, dplyr, jsonlite